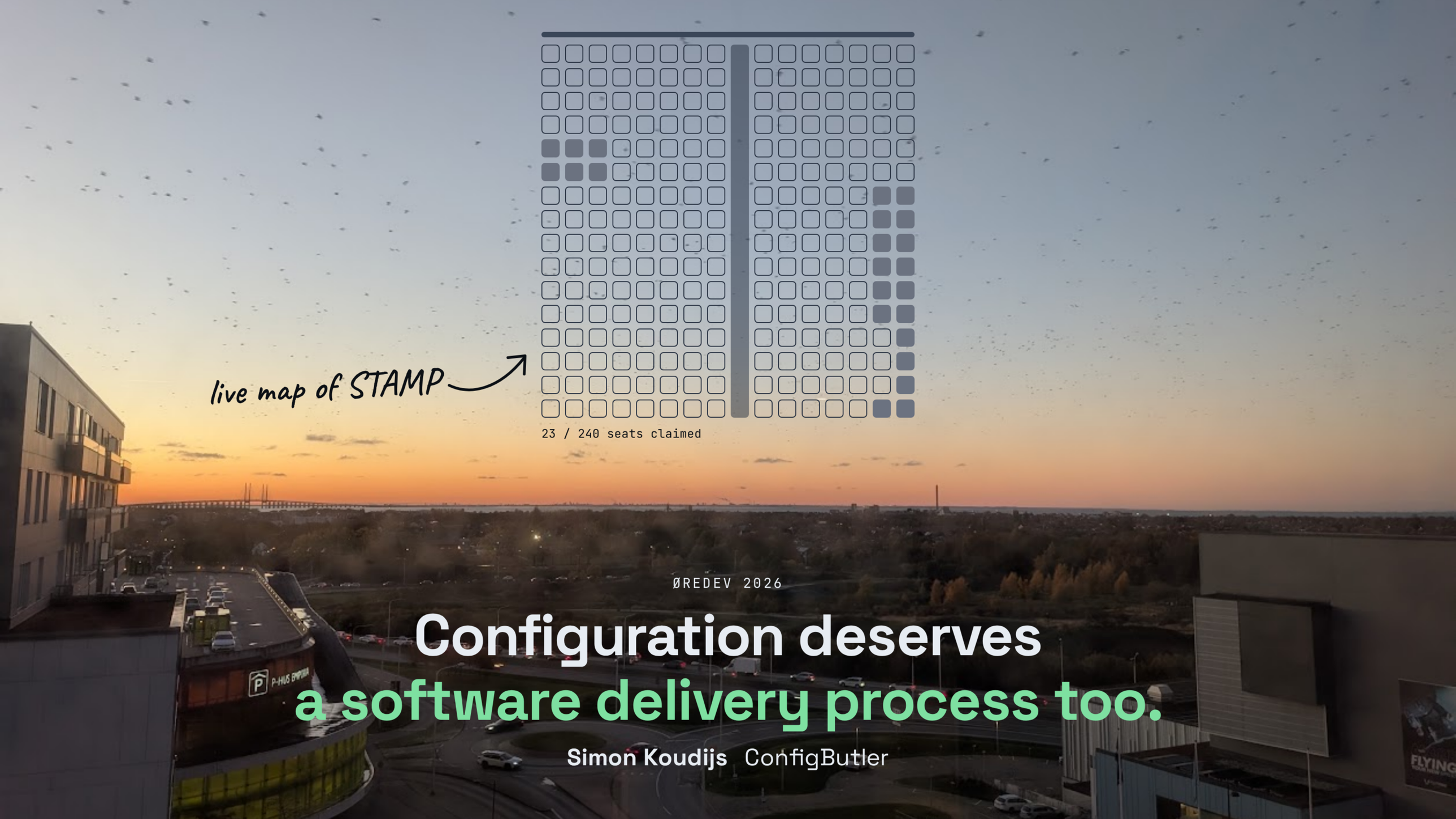


ØREDEV 2026

Configuration deserves a software delivery process too.

Simon Koudijs ConfigButler

TLDR;

slides at: **koudijs.dev/speaking**

Package settings into an image.

Settings screen → Files in Git → **Image** → Test → Production

Model config text-based

Desired state, format choice is yours: YAML, XML, INI, JPG...

A clear place to point at

What is production running? This digest.

Promoted like code

Workloads now get 2:
executable image
config image

There is lots of room for refinement: the core works.

LINKS

- **GitOps Reverser** — github.com/ConfigButler/gitops-reverser
- **settings-image** — github.com/ConfigButler/settings-image
- **Reverse GitOps** — reversegitops.dev
- **Config Deserves Better** — configdeservesbetter.dev

What if we could make the **configuration API** so good,

and store our settings in an **audit log** so good,

that people lose the tendency to build tools in front of it?

Infra as code? Let’s go for infra as data.

TO BE CLEAR

This talk is not a bash on:

- config-as-code
- databases
- legacy applications

But I do dislike it if I just can't **hunt back what has been changing my system.**

Simon Koudijs

`[Software|Product|Cloud|AI] engineer`

- From the Netherlands
- Left my engineering job in 2025 to build my own company
- Frustrated by settings pages: do we still build that ourselves?
- **koudijs.dev**: consultancy, training, speaking
- **ConfigButler**: the hosted product of this idea

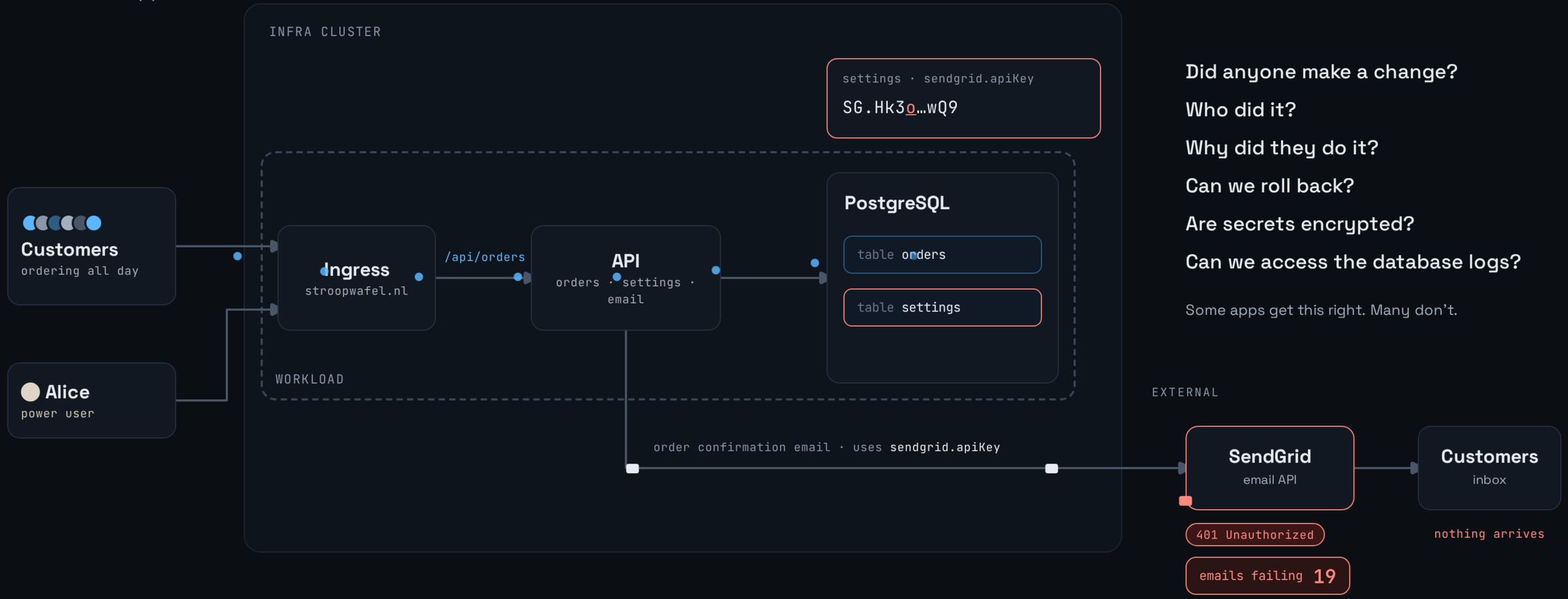
code + config + infra

= production

THE PROBLEM

One typo. No more customer emails.

Settings often live in the app's database, next to the orders. One small edit can break production, and it is not always clear what happened.



Did anyone make a change?

Who did it?

Why did they do it?

Can we roll back?

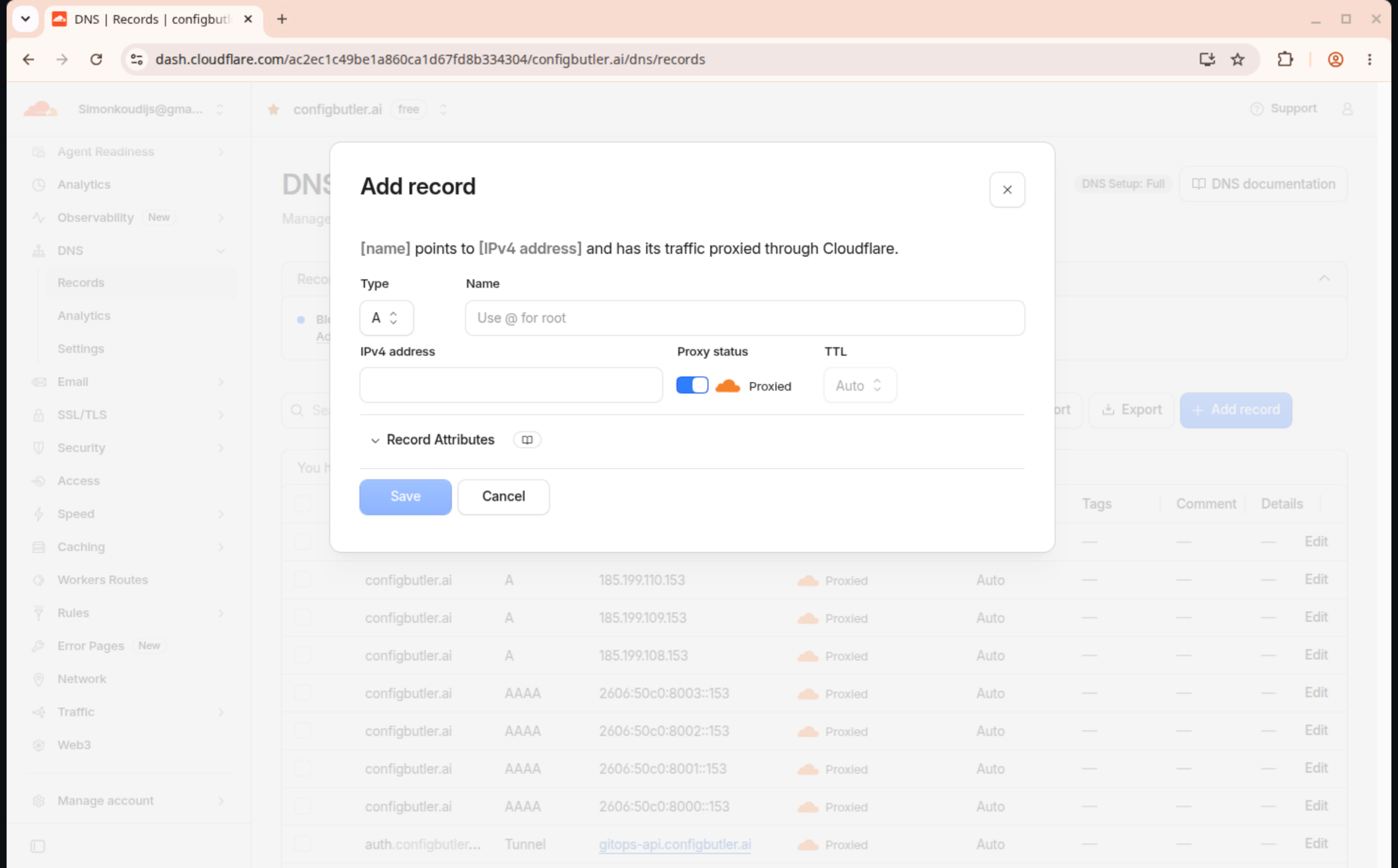
Are secrets encrypted?

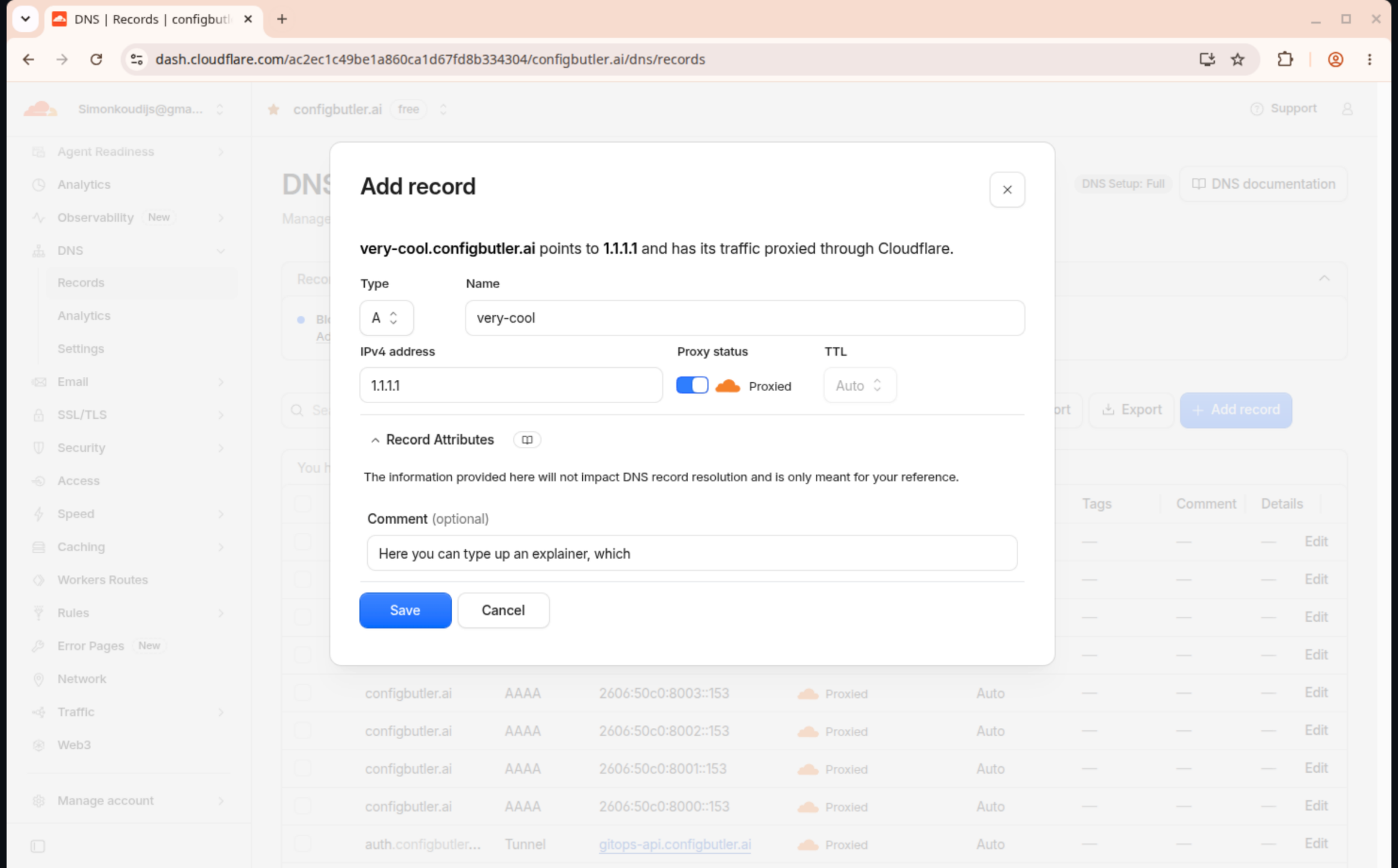
Can we access the database logs?

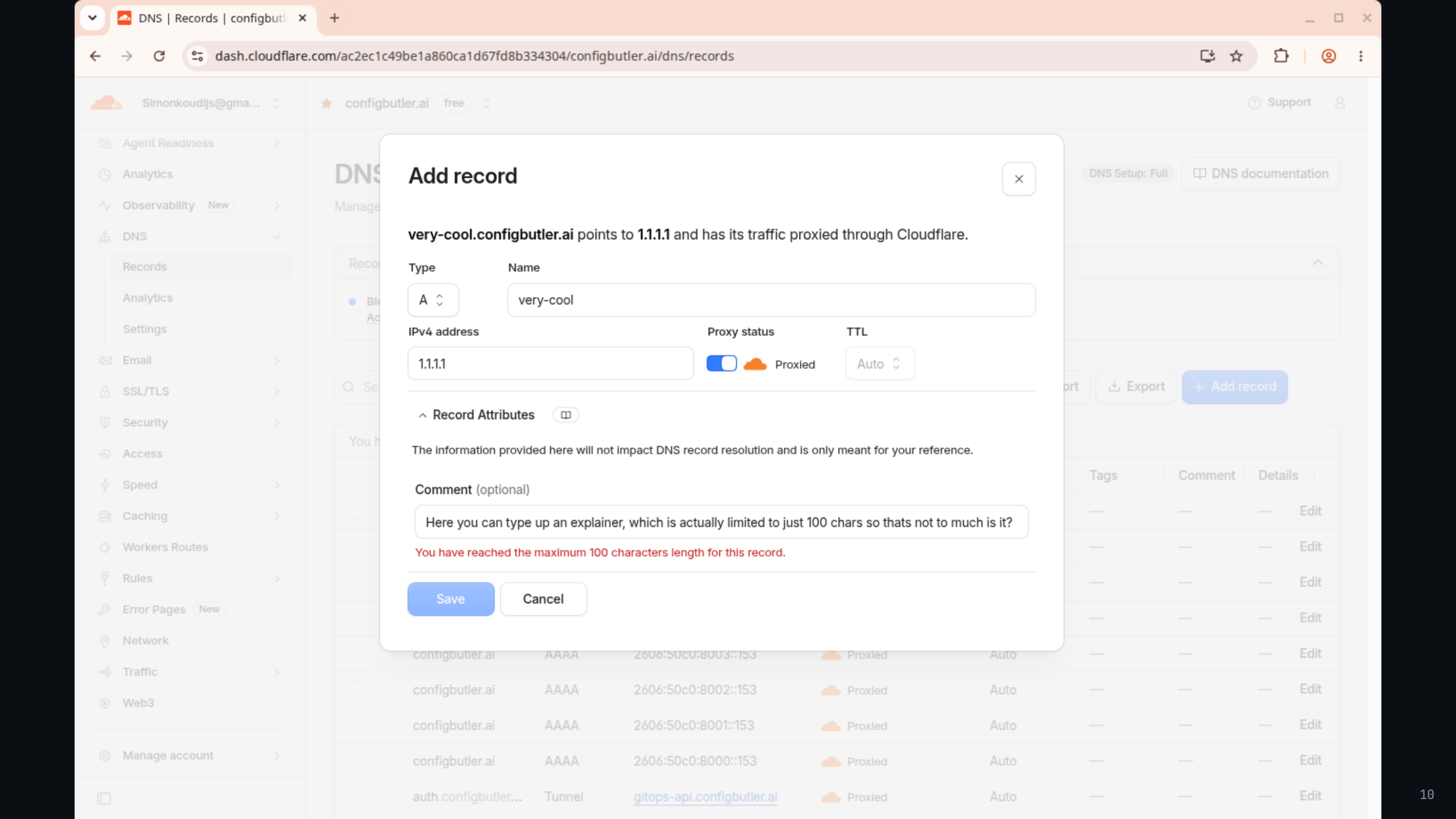
Some apps get this right. Many don't.

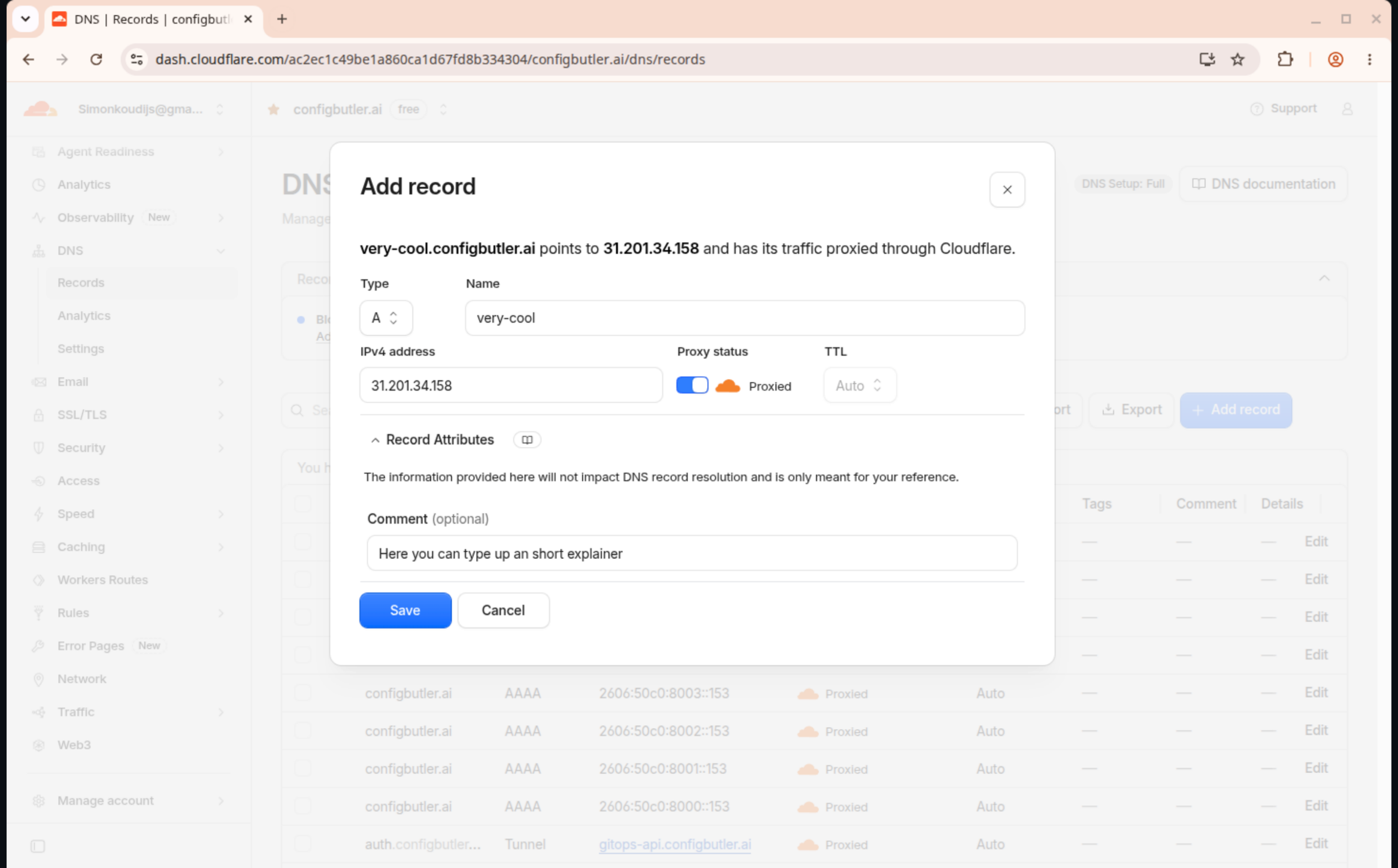
04 Some apps handle this well. Many don't, and you find out in production.

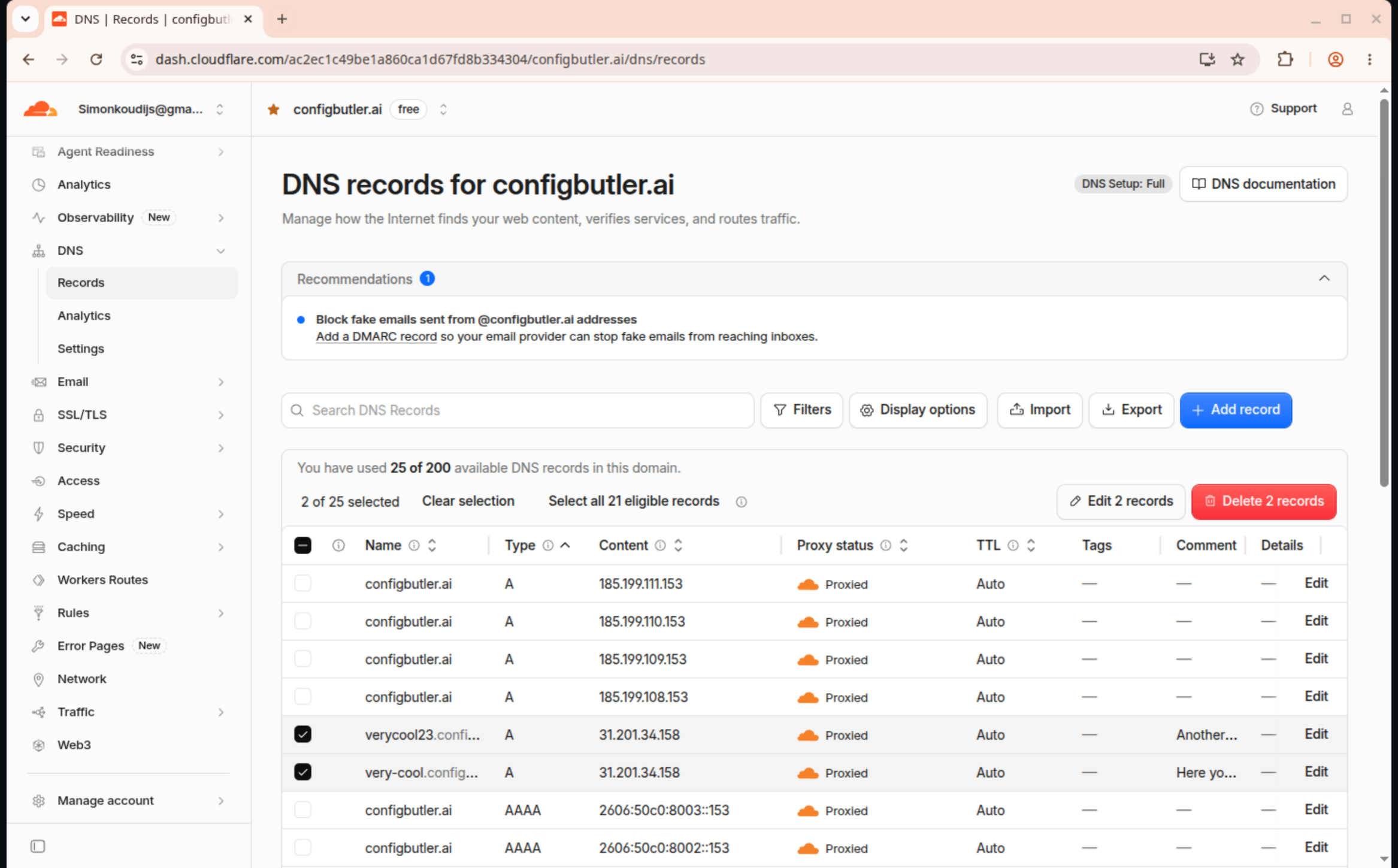
01 emails flow · 02 one typo · 03 silent failure · 04 no answers

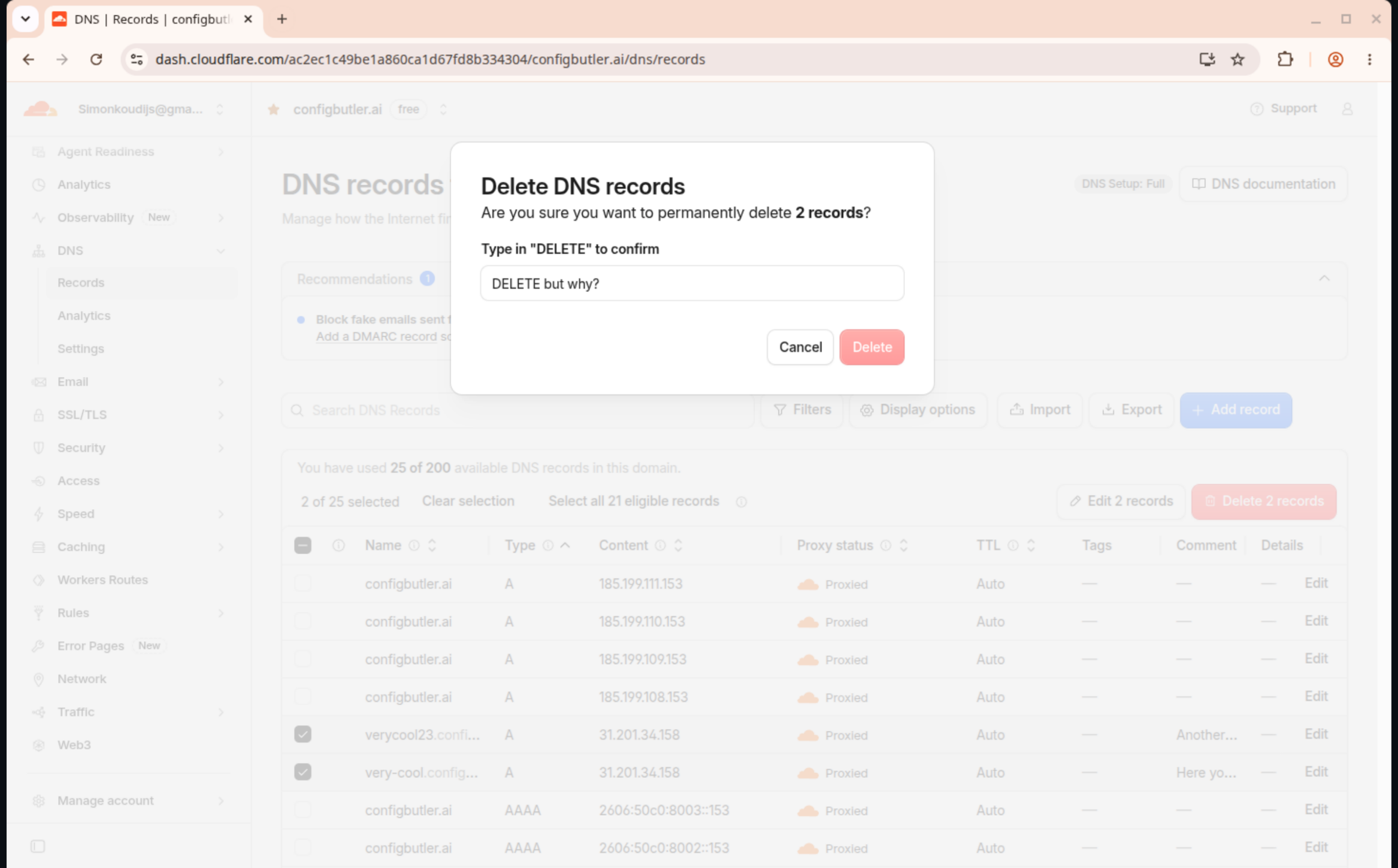

DNS | Records | configbutl
dash.cloudflare.com/ac2ec1c49be1a860ca1d67fd8b334304/configbutler.ai/dns/records
Simonkoudijs@gma...
configbutler.ai
free
Support
Agent Readiness
Analytics
Observability
New
DNS
Records
Analytics
Settings
Email
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
Error Pages
New
Network
Traffic
Web3
Manage account
DNS
Manage
DNS Setup: Full
DNS documentation
Export
Add record
Tags
Comment
Details
Add record
[name] points to [IPv4 address] and has its traffic proxied through Cloudflare.
Type
A
Name
Use @ for root
IPv4 address
Proxy status
Proxied
TTL
Auto
Record Attributes
Save
Cancel
configbutler.ai A 185.199.110.153 Proxied Auto Edit
configbutler.ai A 185.199.109.153 Proxied Auto Edit
configbutler.ai A 185.199.108.153 Proxied Auto Edit
configbutler.ai AAAA 2606:50c0:8003::153 Proxied Auto Edit
configbutler.ai AAAA 2606:50c0:8002::153 Proxied Auto Edit
configbutler.ai AAAA 2606:50c0:8001::153 Proxied Auto Edit
configbutler.ai AAAA 2606:50c0:8000::153 Proxied Auto Edit
auth.configbutler... Tunnel gitops-api.configbutler.ai Proxied Auto Edit

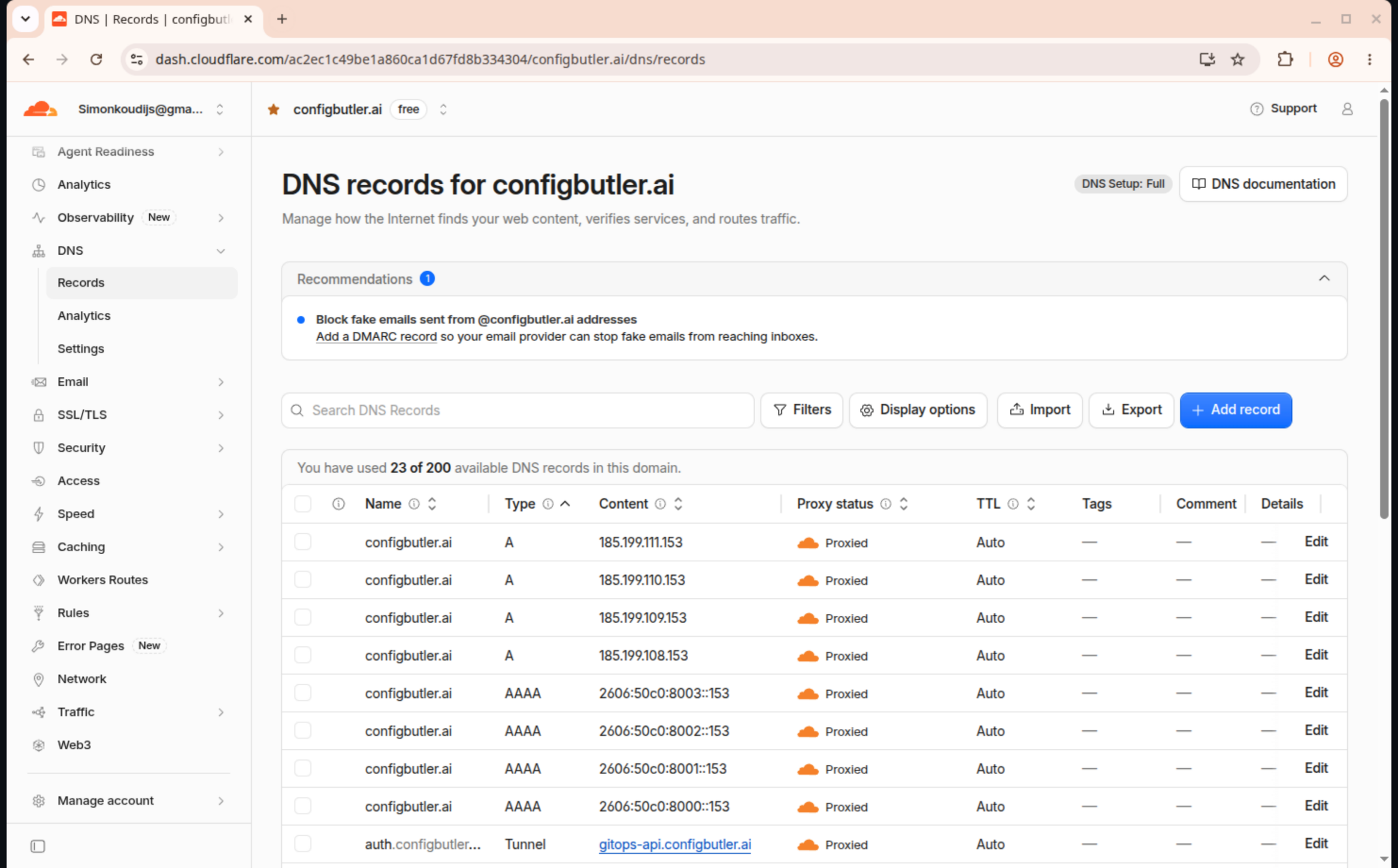

DNS | Records | configbutl
dash.cloudflare.com/ac2ec1c49be1a860ca1d67fd8b334304/configbutler.ai/dns/records
Simonkoudijs@gma...
configbutler.ai
free
Support
Agent Readiness
Analytics
Observability
New
DNS
Records
Analytics
Settings
Email
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
Error Pages
New
Network
Traffic
Web3
Manage account
DNS
Manage
DNS Setup: Full
DNS documentation
Export
Add record
Tags
Comment
Details
Edit
Add record
very-cool.configbutler.ai points to 1.1.1.1 and has its traffic proxied through Cloudflare.
Type
A
Name
very-cool
IPv4 address
1.1.1.1
Proxy status
Proxied
TTL
Auto
Record Attributes
The information provided here will not impact DNS record resolution and is only meant for your reference.
Comment (optional)
Here you can type up an explainer, which
Save
Cancel
configbutler.ai AAAA 2606:50c0:8003::153 Proxied Auto
configbutler.ai AAAA 2606:50c0:8002::153 Proxied Auto
configbutler.ai AAAA 2606:50c0:8001::153 Proxied Auto
configbutler.ai AAAA 2606:50c0:8000::153 Proxied Auto
auth.configbutler... Tunnel gitops-api.configbutler.ai Proxied Auto


DNS | Records | configbutl
dash.cloudflare.com/ac2ec1c49be1a860ca1d67fd8b334304/configbutler.ai/dns/records
Simonkoudijs@gma...
configbutler.ai
free
Support
Agent Readiness
Analytics
Observability
New
DNS
Records
Analytics
Settings
Email
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
Error Pages
New
Network
Traffic
Web3
Manage account
DNS Setup: Full
DNS documentation
Export
Add record
Add record
very-cool.configbutler.ai points to 1.1.1.1 and has its traffic proxied through Cloudflare.
Type
A
Name
very-cool
IPv4 address
1.1.1.1
Proxy status
Proxied
TTL
Auto
Record Attributes
The information provided here will not impact DNS record resolution and is only meant for your reference.
Comment (optional)
Here you can type up an explainer, which is actually limited to just 100 chars so thats not to much is it?
You have reached the maximum 100 characters length for this record.
Save
Cancel
Tags
Comment
Details
configbutler.ai AAAA 2606:50c0:8003::153 Proxied Auto Edit
configbutler.ai AAAA 2606:50c0:8002::153 Proxied Auto Edit
configbutler.ai AAAA 2606:50c0:8001::153 Proxied Auto Edit
configbutler.ai AAAA 2606:50c0:8000::153 Proxied Auto Edit
auth.configbutler... Tunnel gitops-api.configbutler.ai Proxied Auto Edit


DNS | Records | configbutl
dash.cloudflare.com/ac2ec1c49be1a860ca1d67fd8b334304/configbutler.ai/dns/records
Simonkoudijs@gma...
configbutler.ai
free
Support
Agent Readiness
Analytics
Observability
New
DNS
Records
Analytics
Settings
Email
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
Error Pages
New
Network
Traffic
Web3
Manage account
DNS Setup: Full
DNS documentation
Export
Add record
Tags
Comment
Details
Edit
Add record
very-cool.configbutler.ai points to 31.201.34.158 and has its traffic proxied through Cloudflare.
Type
A
Name
very-cool
IPv4 address
31.201.34.158
Proxy status
Proxied
TTL
Auto
Record Attributes
The information provided here will not impact DNS record resolution and is only meant for your reference.
Comment (optional)
Here you can type up an short explainer
Save
Cancel
configbutler.ai AAAA 2606:50c0:8003::153 Proxied Auto
configbutler.ai AAAA 2606:50c0:8002::153 Proxied Auto
configbutler.ai AAAA 2606:50c0:8001::153 Proxied Auto
configbutler.ai AAAA 2606:50c0:8000::153 Proxied Auto
auth.configbutler... Tunnel gitops-api.configbutler.ai Proxied Auto


DNS | Records | configbutl
dash.cloudflare.com/ac2ec1c49be1a860ca1d67fd8b334304/configbutler.ai/dns/records
Simonkoudijs@gma...
configbutler.ai
free
Support
Agent Readiness
Analytics
Observability
New
DNS
Records
Analytics
Settings
Email
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
Error Pages
New
Network
Traffic
Web3
Manage account
DNS records for configbutler.ai
Manage how the Internet finds your web content, verifies services, and routes traffic.
DNS Setup: Full
DNS documentation
Recommendations
1
Block fake emails sent from @configbutler.ai addresses
Add a DMARC record so your email provider can stop fake emails from reaching inboxes.
Search DNS Records
Filters
Display options
Import
Export
Add record
You have used 25 of 200 available DNS records in this domain.
2 of 25 selected
Clear selection
Select all 21 eligible records
Edit 2 records
Delete 2 records
Name
Type
Content
Proxy status
TTL
Tags
Comment
Details
configbutler.ai A 185.199.111.153 Proxied Auto — — — Edit
configbutler.ai A 185.199.110.153 Proxied Auto — — — Edit
configbutler.ai A 185.199.109.153 Proxied Auto — — — Edit
configbutler.ai A 185.199.108.153 Proxied Auto — — — Edit
verycool23.confi... A 31.201.34.158 Proxied Auto — Another... — Edit
very-cool.config... A 31.201.34.158 Proxied Auto — Here yo... — Edit
configbutler.ai AAAA 2606:50c0:8003::153 Proxied Auto — — — Edit
configbutler.ai AAAA 2606:50c0:8002::153 Proxied Auto — — — Edit


DNS | Records | configbutl
dash.cloudflare.com/ac2ec1c49be1a860ca1d67fd8b334304/configbutler.ai/dns/records
Simonkoudijs@gma...
configbutler.ai
free
Support
Agent Readiness
Analytics
Observability
New
DNS
Records
Analytics
Settings
Email
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
Error Pages
New
Network
Traffic
Web3
Manage account
DNS records
Manage how the Internet fir
DNS Setup: Full
DNS documentation
Recommendations
Block fake emails sent f
Add a DMARC record sc
Search DNS Records
Filters
Display options
Import
Export
Add record
You have used 25 of 200 available DNS records in this domain.
2 of 25 selected
Clear selection
Select all 21 eligible records
Edit 2 records
Delete 2 records
Name
Type
Content
Proxy status
TTL
Tags
Comment
Details
configbutler.ai A 185.199.111.153 Proxied Auto Edit
configbutler.ai A 185.199.110.153 Proxied Auto Edit
configbutler.ai A 185.199.109.153 Proxied Auto Edit
configbutler.ai A 185.199.108.153 Proxied Auto Edit
verycool23.confi... A 31.201.34.158 Proxied Auto Another... Edit
very-cool.config... A 31.201.34.158 Proxied Auto Here yo... Edit
configbutler.ai AAAA 2606:50c0:8003::153 Proxied Auto Edit
configbutler.ai AAAA 2606:50c0:8002::153 Proxied Auto Edit
Delete DNS records
Are you sure you want to permanently delete 2 records?
Type in "DELETE" to confirm
DELETE but why?
Cancel
Delete


DNS | Records | configbutl
dash.cloudflare.com/ac2ec1c49be1a860ca1d67fd8b334304/configbutler.ai/dns/records
Simonkoudijs@gma...
configbutler.ai
free
Support
Agent Readiness
Analytics
Observability
New
DNS
Records
Analytics
Settings
Email
SSL/TLS
Security
Access
Speed
Caching
Workers Routes
Rules
Error Pages
New
Network
Traffic
Web3
Manage account
DNS records for configbutler.ai
Manage how the Internet finds your web content, verifies services, and routes traffic.
DNS Setup: Full
DNS documentation
Recommendations
1
Block fake emails sent from @configbutler.ai addresses
Add a DMARC record so your email provider can stop fake emails from reaching inboxes.
Search DNS Records
Filters
Display options
Import
Export
Add record
You have used 23 of 200 available DNS records in this domain.
Name
Type
Content
Proxy status
TTL
Tags
Comment
Details
configbutler.ai A 185.199.111.153 Proxied Auto — — — Edit
configbutler.ai A 185.199.110.153 Proxied Auto — — — Edit
configbutler.ai A 185.199.109.153 Proxied Auto — — — Edit
configbutler.ai A 185.199.108.153 Proxied Auto — — — Edit
configbutler.ai AAAA 2606:50c0:8003::153 Proxied Auto — — — Edit
configbutler.ai AAAA 2606:50c0:8002::153 Proxied Auto — — — Edit
configbutler.ai AAAA 2606:50c0:8001::153 Proxied Auto — — — Edit
configbutler.ai AAAA 2606:50c0:8000::153 Proxied Auto — — — Edit
auth.configbutler... Tunnel gitops-api.configbutler.ai Proxied Auto — — — Edit

Audit logs | Simonkoudijs@

dash.cloudflare.com/ac2ec1c49be1a860ca1d67fd8b334304/audit-log?action_type~neq=view

Simonkoudijs@gma...

Audit logs

Support

configbutler.ai / DNS

configbutler.ai
Domains

Audit logs
Manage account

Analytics
koudijs.dev / DNS

Observability New

Email

SSL/TLS

Web3

Manage account

Members

Billing

Account API tokens

Audit logs

Alerts

Blocked content

Abuse reports

Carbon Impact Report

Configurations

Tagged Resources Beta

Actio... ≠ view × + Add filter Clear filters

Last 7 days

Actions Over Time

25 Success 0 Failed

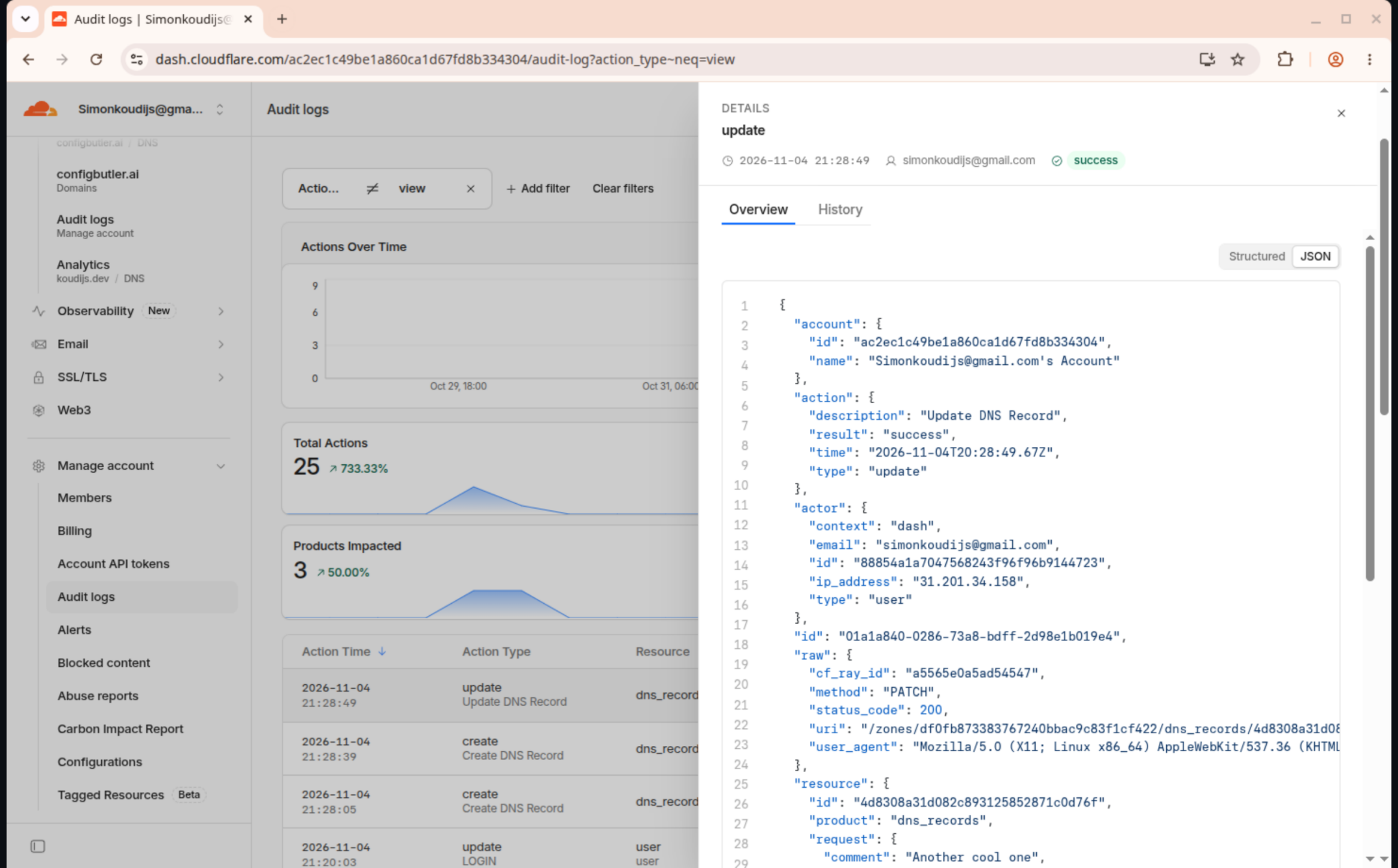
Total Actions
25 ↗ 733.33%

Unique Actors
3 ↗ 200.00%

Products Impacted
3 ↗ 50.00%

Failure Rate
0 %

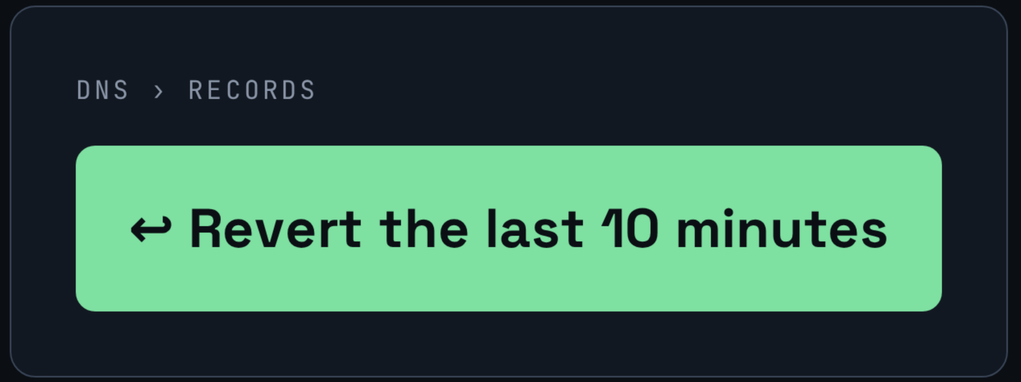
Action Time	Action Type	Resource	Actor Email	Actor Context	Zone Name
2026-11-04 21:28:49	update Update DNS Record	dns_records	simonkoudijs@gmail.com User	Dashboard	configbutler.ai
2026-11-04 21:28:39	create Create DNS Record	dns_records	simonkoudijs@gmail.com User	Dashboard	configbutler.ai
2026-11-04 21:28:05	create Create DNS Record	dns_records	simonkoudijs@gmail.com User	Dashboard	configbutler.ai
2026-11-04 21:20:03	update LOGIN	user user	simonkoudijs@gmail.com User	API (no credential detail)	-

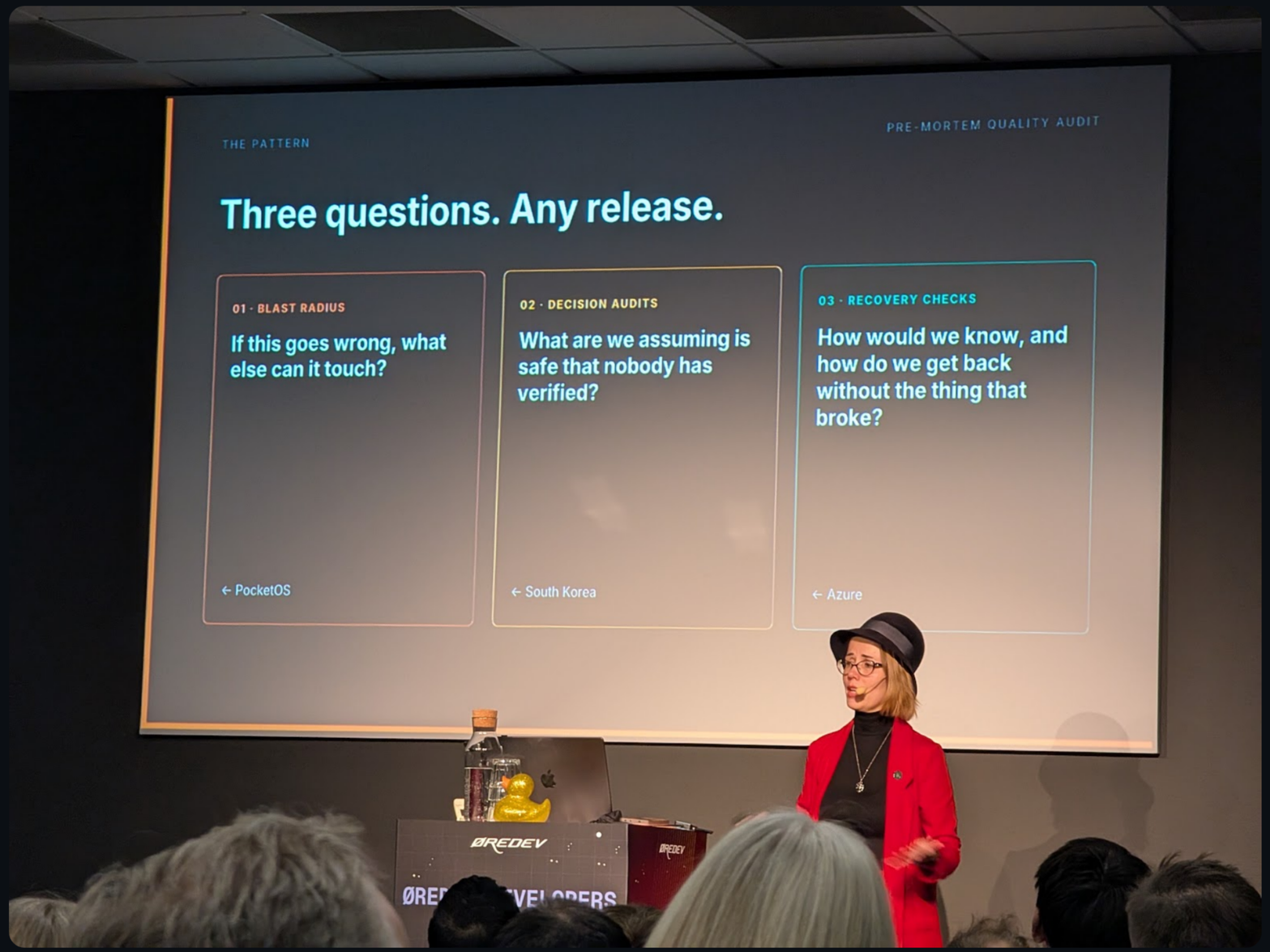

Audit logs | Simonkoudijs
dash.cloudflare.com/ac2ec1c49be1a860ca1d67fd8b334304/audit-log?action_type~neq=view
Simonkoudijs@gma...
Audit logs
configbutler.ai
Domains
Audit logs
Manage account
Analytics
koudijs.dev / DNS
Observability New
Email
SSL/TLS
Web3
Manage account
Members
Billing
Account API tokens
Audit logs
Alerts
Blocked content
Abuse reports
Carbon Impact Report
Configurations
Tagged Resources Beta
Actio... ≠ view
+ Add filter
Clear filters
Actions Over Time
9
6
3
0
Oct 29, 18:00
Total Actions
25 ↗ 733.33%
Products Impacted
3 ↗ 50.00%
Action Time
Action Type
Resource
2026-11-04 21:28:49
update
Update DNS Record
dns_record
2026-11-04 21:28:39
create
Create DNS Record
dns_record
2026-11-04 21:28:05
create
Create DNS Record
dns_record
2026-11-04 21:20:03
update
LOGIN
user
user
DETAILS
update
2026-11-04 21:28:49
simonkoudijs@gmail.com
success
Overview
History
Structured
JSON
{
"account": {
"id": "ac2ec1c49be1a860ca1d67fd8b334304",
"name": "Simonkoudijs@gmail.com's Account"
},
"action": {
"description": "Update DNS Record",
"result": "success",
"time": "2026-11-04T20:28:49.67Z",
"type": "update"
},
"actor": {
"context": "dash",
"email": "simonkoudijs@gmail.com",
"id": "88854a1a7047568243f96f96b9144723",
"ip_address": "31.201.34.158",
"type": "user"
},
"id": "01a1a840-0286-73a8-bdff-2d98e1b019e4",
"raw": {
"cf_ray_id": "a5565e0a5ad54547",
"method": "PATCH",
"status_code": 200,
"uri": "/zones/df0fb873383767240bbac9c83f1cf422/dns_records/4d8308a31d08
"user_agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML
},
"resource": {
"id": "4d8308a31d082c893125852871c0d76f",
"product": "dns_records",
"request": {
"comment": "Another cool one",

IT HAPPENS TO ME TOO

My own Cloudflare DNS change

- I could have written a better runbook
- I could have scripted the right values
- **But when I'm panicking, why isn't there a button for this?**





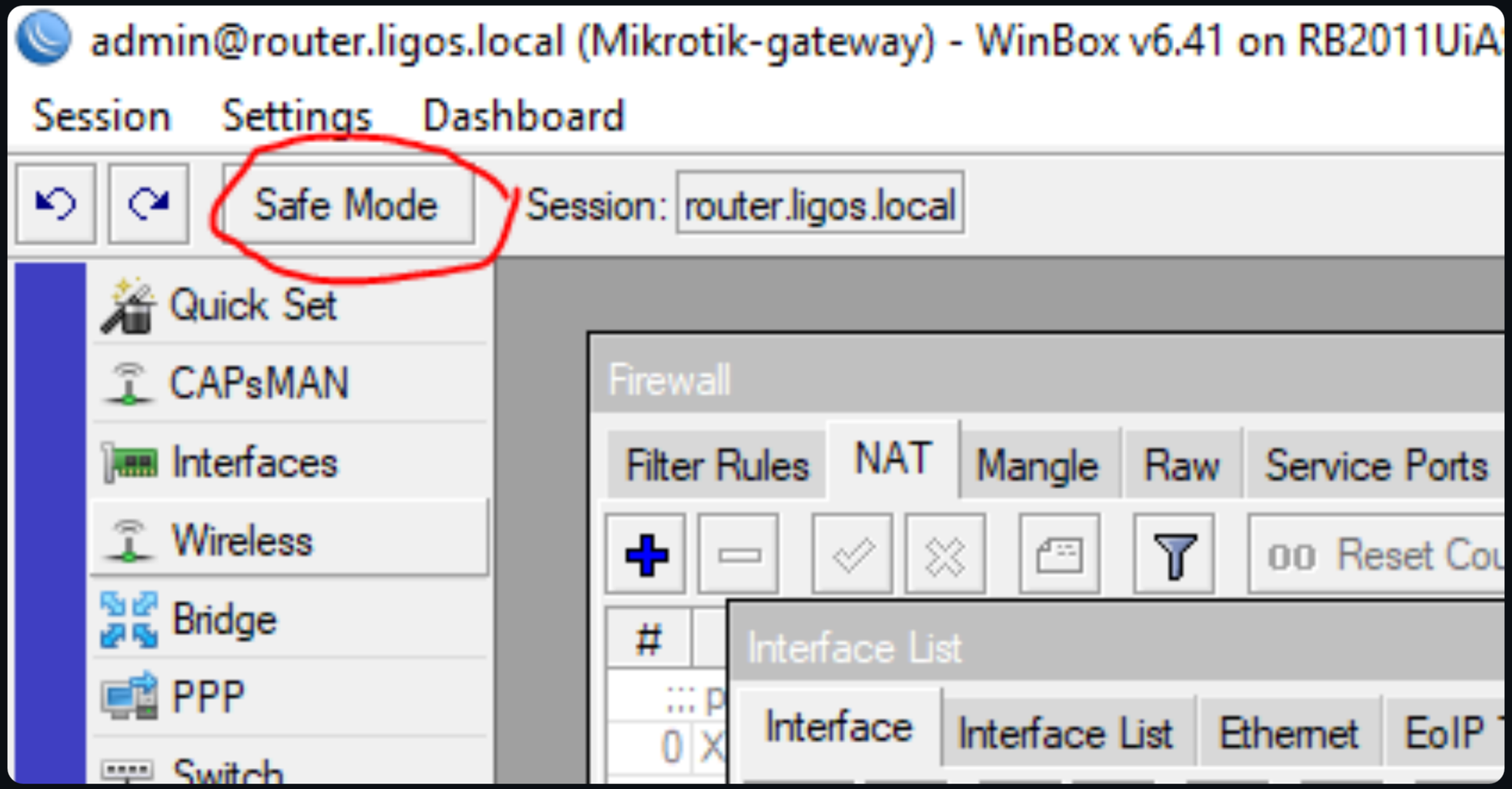
YESTERDAY, AT ØREDEV

Christine Pinto

A talk on outages.

> **“We don’t need to test that.”**

Legacy



SOURCE

Making Mikrotik Safe
blog.ligos.net/2018-02-22/Making-Mikrotik-Safe.html

Where this applies

This does NOT apply to:

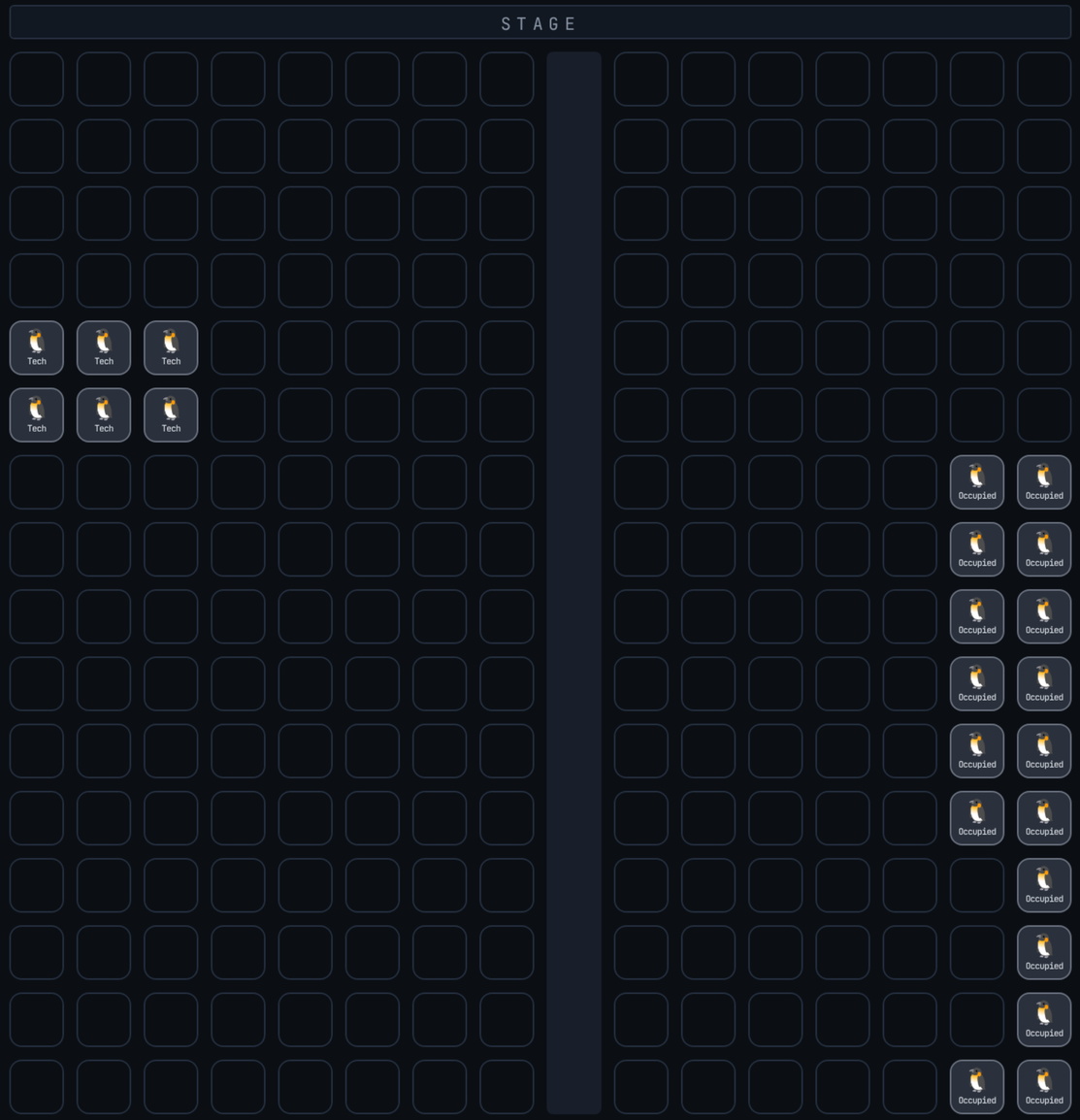
- Transactional data like new orders

This does apply to:

- Your legacy business app with a very old screen: it probably can read a file, can't it?
- Any software that needs settings and that would rather just define the things it needs

Think of it as DI for config on steroids.

WHAT YOU JUST DID



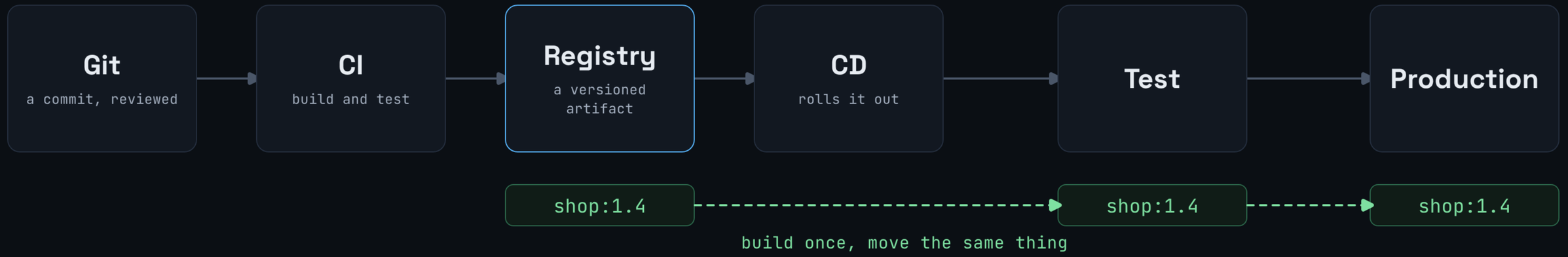
23 / 240 seats claimed

WHAT YOU JUST DID · IN GIT

Your commits. Your names, your reasons.

```
$ cd /workspaces/talks/external/k8s-trail
$ git pull --ff-only
$ git --no-pager log --invert-grep --grep='^chore' --format='%C(cyan)%<(16,trunc)%an%C(reset) %s' -15
$ git --no-pager log -1 --invert-grep --grep='^chore' --format=fuller
```

You know this picture



Your ingredients will differ. It all depends.

03 I don't know your exact system, and that's fine: the shape is what counts.

THE SAME ROAD, FOR CONFIGURATION

A second stream next to the code

03 The pod states exactly what it runs: this version of the code, and this version of the settings.

LET'S APPLY THIS

Back to text files. Oef, two steps back?

A specification

What it should be

```
row: 6
column: 6
claimedBy: simon
color: "#a8e05f"
```

- **Only the desired state.** Nothing that happened at runtime
- **Several small files that reference each other,** for what's complex
- **A special place for secret values.** We don't want them in sight

Oh dear, what is he proposing?

But what about my nice relational database?

And how do you handle merge conflicts?

Come see that tomorrow. I do advise you to have a temporary database in front of it, and a very nice API.

ONE CHAIR, IN ANY FORMAT

It’s all text-based files.
What format do you like?

seat.ini

```
[seat]
row = 6
column = 6
color = "#a8e05f"
```

seat.xml

```
<seat row="6" column="6">
  <color>#a8e05f</color>
</seat>
```

seat.json

```
{
  "row": 6,
  "column": 6,
  "color": "#a8e05f"
}
```

seat.yaml

```
row: 6
column: 6
color: "#a8e05f"
```

simon.jpg

even this

↓

`shop-settings:43` just the format your app reads

Do you have a good option to create these artifacts?

DO YOU HAVE A GOOD OPTION?

Creating the artifacts

A Dockerfile

```
FROM scratch
COPY settings/ /
```

You already know it. No base image and no shell.

ORAS

```
oras push \
  registry/shop-settings:43 \
  settings.yaml
```

Pushes plain files to any OCI registry.

settings-image

The same files give the same digest. Nothing in it executes. A Secret must be SOPS-encrypted, or the build stops.

Mine, and still v0.1.

OPEN SOURCE

settings-image
github.com/ConfigButler/settings-image

ORAS
oras.land

TOMORROW, 11:15, TURING

(Ab)using the Kubernetes API as a Configuration Backend

- More detail on **GitOps**, **Reverse GitOps**, and even **bi-directional GitOps**
- How can you run this **at scale**?
- Why is Kubernetes a good fit for **modelling your configuration**?

Don’t want to run Kubernetes? It’s useful for you too. Self-hosting is fine, but managed solutions should arise soon.

FOR THE STRUCTURED ONES

One shape: KRM

```yaml
apiVersion: examples.configbutler.ai/v1alpha1
kind: Seat
metadata:
  name: ordev-1-stamp-6-6
spec:
  row: 6
  column: 6
  color: '#a8e05f'
  emoji: 🐧
```

apiVersion
which schema it follows

kind
what it is

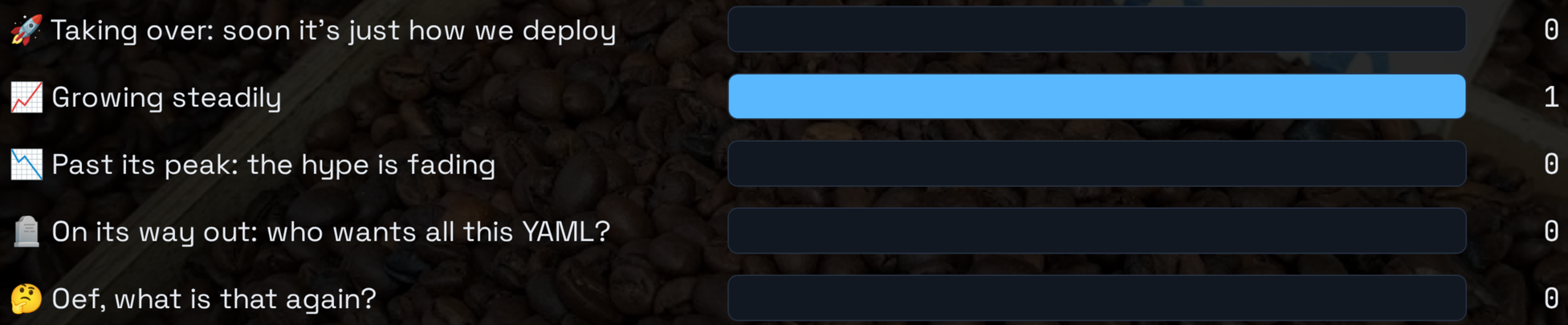
metadata
what it's called

spec
the desired state

The Kubernetes Resource Model: a description, in a shape many tools already read.

WHAT YOU ANSWERED AT THE DOOR

Where is GitOps going?

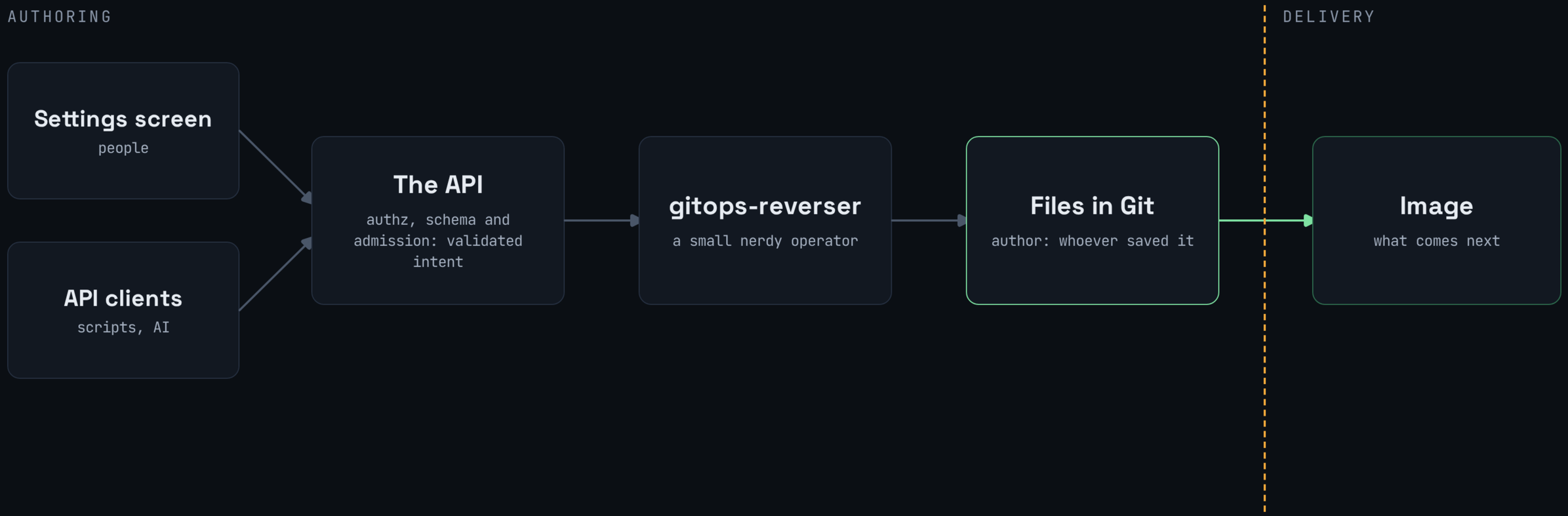


1 answered

> **End users don't need to be in Git.**
> **But their actions should be.**

NOW THAT WE HAVE FILES

Your users never see Git. You still get the log.



03 Authoring ends here, and delivery starts here. Saved doesn't mean live any more.

SOME OF YOU RECOGNISED THIS

Don’t dismiss this if you dislike Kubernetes.

1

The idea stands

You can push a representation of your configuration as an artifact, however you produce it.

2

Be explicit about the process around it

Who changes it, what checks it, and how it moves to production.

The magic behind it: tomorrow, 11:15, Turing. (Ab)using the Kubernetes API as a Configuration Backend.

THE MAIN POINT

Any file. One image.

The complete model

of the current desired state, as your end users configured it

A clear place to point at

“What is production running?”
This digest.

04 One line answers what’s running. It also answers what was running an hour ago.

01 any file · 02 an image · 03 the model · 04 point at it

DEMO

One secret word.
Let’s follow it to the pod.

settings-image github.com/ConfigButler/settings-image | **Flux** fluxcd.io/flux/components/image | **zot** zotregistry.dev

WHAT IS HAPPENING RIGHT NOW

Ship settings like images.

Settings ship as an image. Flux picks up new versions, and the Deployment mounts them as a read-only image volume.

Inside orderer-config:43 · mounted at /etc/orderer/config

```
/etc/orderer/config/
├── emailsettings/order-confirmations.yaml
├── apikey/sendgrid.yaml                    # spec only, no status
└── secret/sendgrid-api-key.sops.yaml       # still SOPS-encrypted

# emailsettings/order-confirmations.yaml
spec: { sender: orders@stroopwafel.nl, templateId: order-confirmed, apiKeyRef: sendgrid }
```

The app · just reads YAML, through a typed lib generated from the CRDs

```
cfg := ordererconfig.Load("/etc/orderer/config")      // generated from the CRDs
email := cfg.EmailSettings("order-confirmations")
key := cfg.ApiKey(email.APIKeyRef).Decrypt(ctx)   // workload identity → KMS
mailer.Send(email.Sender, email.TemplateID, key, order)

// no API calls, no database: settings are just files
```

GitOps repo · the real truth · orderer's Deployment

flux orderer-config 42 → 43

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orderer
spec:
  template:
    spec:
      containers:
      - name: orderer
        image: ghcr.io/stroopwafel/orderer:1.4
        volumeMounts:
        - name: config
          mountPath: /etc/orderer/config
      volumes:
      - name: config
        image:  # a container image as a volume
          reference: ghcr.io/stroopwafel/orderer-config:43@sha256:90df… # {"$imagepolicy": "flux:config"}
          pullPolicy: IfNotPresent
```

Cluster follows the GitOps repo — new pod mounts orderer-config:43

05 And the app just reads YAML files. A typed library, generated from the CRDs, makes that a few lines of code.

01 publish | 02 registry | 03 Flux bumps | 04 mounted | 05 the app reads it

coffee-settings github.com/ConfigButler/coffee-settings

settings-image github.com/ConfigButler/settings-image

DEMO · THE SECRET WORD ARRIVED

coffee-settings.k8s.koudijs.dev/decrypted/ ↗

From a commit to a pod.

```
$ cd /workspaces/talks/external/coffee-settings
$ git pull --ff-only
$ git --no-pager log --format='%C(yellow)%h%C(reset) %C(cyan)%<(16,trunc)%an%C(reset) %s' -6
$ grep -E '^(kind|data)|name:|apiKey' coffee/resources/coffee-mail.sops.yaml | cut -c1-72
$ crane export cr.k8s.koudijs.dev/configbutler/coffee-settings:main - | tar t
$ cd /workspaces/talks/external/k8s && git fetch -q
$ git log origin/main --author=flux --format='%h %s' -3 -- k8s.koudijs.dev/2-gitops/coffee-settings | cut -c1-120
$ kubectl -n coffee-settings get deploy coffee-settings -o yaml | grep -m1 settings-reference
$ curl -s https://coffee-settings.k8s.koudijs.dev/decrypted/STATUS.txt
```

settings-image github.com/ConfigButler/settings-image | **Flux** fluxcd.io/flux/components/image | **Kargo** kargo.akuity.io

PROMOTE IT

Move the digest. Not the settings.

03 Something's wrong? Point at the previous digest. That's the button I wanted in my Cloudflare panic.

01 test first — 02 promote — 03 go back

SOPS getsops.io | age age-encryption.org | OpenBao openbao.org

WHAT JUST HAPPENED TO THAT WORD

Ciphertext until the very last moment.

Git
ciphertext

→

CI
ciphertext, holds no key

→

Registry
ciphertext

→

Your pod
opened in memory, with its own identity

```
kind: Secret
metadata:
    name: coffee-mail                    # the name stays readable
data:
    apiKey: ENC[AES256_GCM,data:gxfeGP4qKRyhDE…]
```

A registry is not a secret store. **But the API key from the start now has a version too.**

Her typo is now a commit.

- **Who did it?** The commit author
- **Roll it back?** `git revert`
- **Where was it checked?** The schema, and the test environment first
- **Prod DB access to see it?** No: it's in Git
- **What is production running?** This digest

NONE OF THESE PARTS ARE EASY

What I haven’t solved

1. **Different values per environment**
   One image per overlay, and you no longer promote the same thing. Values from outside, and the model isn’t complete
2. **Testing it as a whole**
   One file can be checked. Does it work with this code and this infra? That takes a real environment
3. **Code and config moving apart**
   Two streams, two versions. Which combinations are valid?
4. **Going back while the API disagrees**
   Roll back the image, and the settings screen still holds the new value. Which one is the truth?
5. **Secrets over time**
   Rolling back to a key you already revoked doesn’t help

Hard questions. But with a settings table you often can’t even ask them: there is no version to point at.

Not for everything.

Leave it

- Orders and other transactions: that’s what your database is for
- Settings that change every minute

Worth it when

- One bad change can take production down
- You can’t say who changed what, or when
- You can’t say how to go back
- Settings per tenant, at scale

TLDR;

slides at: **koudijs.dev/speaking**

Package settings into an image.

Settings screen → Files in Git → **Image** → Test → Production

Model config text-based

Desired state, format choice is yours: YAML, XML, INI, JPG...

A clear place to point at

What is production running? This digest.

Promoted like code

Workloads now get 2:
executable image
config image

Tomorrow, 11:15, Turing: (Ab)using the Kubernetes API as a Configuration Backend. Is this contrarian? And is there a chance it's right?

LINKS

- **GitOps Reverser** — github.com/ConfigButler/gitops-reverser
- **settings-image** — github.com/ConfigButler/settings-image
- **Reverse GitOps** — reversegitops.dev
- **Config Deserves Better** — configdeservesbetter.dev